

Cleaning Process:

Within the cleaning process I checked that the input files exist and are readable, I also personally went through the file to see the data first hand and any malicious data that could later be cleaned. Removed the hidden Windows carriage-return character `^M/r` from the end of each line. Corrected the typo “mae” to male when looking at the gender row. Replaced the missing survival values with NA to indicate that the data for that specific person/entry is unknown. Removed exact duplicate rows that were contained in the data and also wrote the cleaned data to a new output file instead of having to change the original to keep track of the uncleaned and cleaned data. Also decided to try and use a temp file in case the cleaning steps were going to overwrite the original data. The valid gender values are assumed to be male and female as well and only exact duplicate rows are removed as well. Sed was used because it is good for making simple text substitutions across every line of a file which I found by searching through StackOverflow and finding sed as a command to use for cleaning the data. It was used to remove the hidden carriage-return character `\r` at the end of lines. It was also used to fix consistent data errors, such as changing “mae” to “male”. sed works well for these tasks because the changes are based on recognizable text patterns. The cleaned output from sed was written to a temporary file before duplicate removal.

Reporting Process:

Read the cleaned dataset as the input to the reporting script. Skipped the header row before processing the rest of the passenger data to skip the header row and process the actual data within the CSV file. Used **awk** arrays to store counts and totals for each passenger class and gender. Furthermore, it incremented the passenger and survival counts. Added the ages together. Printed the final report to stdout and printed the

warnings/errors to stderr so we do not have a mixing report. An example of getting the average age I used the formula of (Sum of all valid ages/Number of valid ages) when reporting. `age_sum[class] += age` -> added each passenger's age to the total for that class. `age_count[class]++` -> counts how many passengers had a valid numeric age. Then we can use the final formula `avg_age = age_sum[class] / age_count[class]` to get the result of the average age in the data.